MULTI AGENT DYNAMIC TRAFFIC CONTROL

Gagan Vadlamudi
University at Buffalo
Buffalo, New York
gaganvad@buffalo.edu

Muni Sai Kannemmagari
University at Buffalo
Buffalo, New York
munisaik@buffalo.edu

Sai Sri Kolanu
University at Buffalo
Buffalo, New York
saisriko@buffalo.edu

***Abstract*—Traffic congestion at signalised intersections is a major source of travel delay, fuel waste, and urban air-pollution. Fixed-time or actuated signal plans cannot react fast enough to the volatile, non-stationary demand seen in modern cities. We cast traffic-signal control as a sequential decision-making problem and develop a *multi-agent deep-reinforcement-learning* framework in which every intersection operates as an autonomous learner. The microscopic *CityFlow* simulator, wrapped in a Gym-compatible interface, provides lane-level vehicle dynamics at approximately $20\times$ real time, enabling large-scale experimentation on a 4×4 grid (16 intersections, 72-dimensional observation per node, nine signal phases each). Starting from a single-agent baseline, we progressively integrate algorithmic enhancements—DoubleDQN, Dueling architecture, Graph-Attention convolution for spatial context, and Prioritised Experience Replay—within a centralised-training/decentralised-execution pipeline that shares experience across agents but executes policies locally at 1Hz.**

Across 500 training episodes (1000 steps each) the best variant, Dueling-Double DQN with PER and neighbour-aware rewards, cuts the mean vehicle waiting time from 55.1 s under a random policy to 12.9 s (-76%) and improves network-wide travel time by 17%. Reward normalisation and gradient clipping proved critical for stable learning, while Double DQN markedly reduced over-estimation spikes. The results demonstrate that scalable multi-agent RL can learn cooperative "green waves" that generalise to stochastic traffic patterns without explicit traffic-flow models, offering a viable path toward adaptive signal control in real-world urban grids.

I. Problem Statement

Urban intersections still rely largely on fixed-time signal plans that assume an "average" traffic load, even though real-world demand swings sharply with events, weather, and daily travel habits. The mismatch between static schedules and dynamic flows translates into longer queues, wasted fuel, and avoidable emissions. In this project we frame traffic-signal control as a sequential decision-making problem and ask: Can a learning-based agent discover signal timings that adapt on the fly and outperform hand-tuned schedules? To explore this, we couple the lightweight CityFlow microscopic simulator with a custom Gym-style wrapper that exposes standard reset, step, and render hooks. On top of that interface we implement a Deep Q-Network (DQN) agent—one per intersection—that learns from simulated experience to minimise network-wide vehicle delay. Our objective is to quantify how much such an RL framework can shorten average waiting times compared with traditional fixed-cycle and actuated baselines, while keeping the solution scalable to larger road networks.

II. Environment setup

CityFlow[1] is a microscopic, multi-agent traffic simulator designed for reinforcement-learning research. Because it runs entirely on the CPU and parallelises lane-level updates efficiently, it executes roughly twenty times faster than the industry-standard SUMO while preserving realistic car-following and lane-changing behaviour.

A CityFlow experiment is defined by three JSON files:

- **Roadnet file** –This file defines the structure if the Road network. It consists of multiple intersections and each road may contain multiple lanes. These road links are controlled using traffic signals at intersections.
- **Flow file** – This file defines the traffic flow. We can define multiple features for the vehicle such as max acceleration, max speed, minimum gap to the front vehicle and so on. We can even define the route to be followed by the vehicle, interval and so on.
- **Config file** – Config file is the entry point where we define main parameters. We provide the flow file and Roadnet file in the config file.

Also, the simulation of the environment can be saved in the replay file which can be replayed using the interactive environment provided by the city flow.

A. Gymnasium Wrapper

To plug CityFlow into standard RL pipelines we adopt a public GitHub wrapper[2] that implements the same interface as gymnasium environment. Intersections are defined while initializing the environment. It stores details such as:

1) Number of traffic light phases
2) Incoming and outgoing lanes
3) Directions of each lane group

Some of the key methods in gym wrapper that we used for cityflow environment.

- **__init__(configPath, episodeSteps)**: It parse JSON, build spaces, start simulator. It accepts arbitrary road networks.

- **reset():** Resets the city flow environment with a random seed and returns the initial observation.

- **step(action)**: Step function performs an action given a state. It advances the city flow environment one timestep to reach the next state and calculates the reward based on the waiting time of the vehicles.
  The maximum number of timesteps per episode is defined as 1000.

- **render(mode='human')**: it returns lightweight text output, where full animation handled by CityFlow GUI.

- **_get_observation():** This function get observations of query lane queues and build observation dictionary. Thjis function is internally called by step and reset function.

- **Reward**: Reward is calculated based on the waiting duration of the vehicles. A linear negative penalty is added to each vehicle which is waiting at the intersection in proportion to its waiting time. Our goal here is to minimize the waiting times of the vehicles at the intersections. These rewards are calculated with respect to each intersection as it is crucial for the multi-agent setup.

B. Observation Space:

Observation space is defined as a dictionary with each intersection's observations represented as muti-discrete. For each intersection, observation includes vehicle counts which are waiting at incoming and outgoing lanes with maximum number of vehicles defined initially.

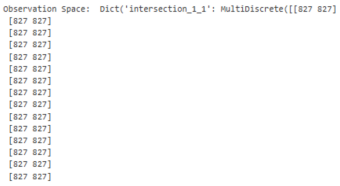

Fig. 1. Observation space

C. Action Space:

Action Space is a multi-discrete array with traffic light phases with each intersection. Each intersection has its own discrete action space reflecting the number of possible traffic light phases.

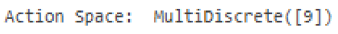

Fig. 2. Action Space

D. Environment visualisation

During early training, we trained on a single-intersection network supplied with the wrapper. Initially, we trained on single intersection which shown in below.

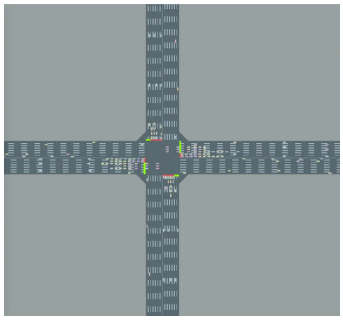

Fig. 3. Single intersection environment with a single agent.

This environment performs random actions sampled from the action space given a state.
For the final experiments we scaled up to a 4 × 4 grid (16 signalised intersections). The wrapper needs only a new roadnet.json and flow.json. We haven't changed the code much. Replays rendered in the CityFlow GUI clearly show coordinated green waves propagating along both axes, validating that multi-agent learning can exploit the network structure.

III. Modelling

Although we have a fixed Roadnet and flow file, by re-seeding the City Flow engine at each episode we re-sample all the stochasticity in car-following, lane-changes, etc., which gives us a simple way to evaluate generalization without changing the high-level demand.

A. Single Agent System

We have implemented DQN[3] on a single intersection of the city flow environment based on the observation space of per lane waiting counts. The Q values are approximated using a simple multi-layer perceptron. The architecture of the neural network has two fully connected hidden layers followed by a normalization layer. This network outputs Q values for all the actions for that state.
The configuration that we have used for training is as follows:

- Batch size: 128
- Replay buffer size: 10 000 transitions (state, action, next state, reward, done)
- Discount factor (γ) : 0.85
- Exploration: ϵ-greedy, $\epsilon_0 = 1.0 \rightarrow \epsilon_{min} = 0.2$ decayed exponentially over 500 episodes
- Training frequency: update network every 4 environment steps
- Target network update: copy weights every 8 episodes.
- Maximum number of steps per Episode: 1000 steps
- Optimizer: Adam with a learning rate of 1e-4.
- Loss: mean-squared error between Bellman target and Q-network output.

The reward is equal to the negative of the total number of vehicles waiting at each intersection which helps the network learn the minimum queue length.

Randomization:

At the start of each episode, we sample an RNG seed for the City Flow engine. This re-draws all the low-level car-following and lane-change randomness. So although the high-level flow schedule is identical, each run is a new "realization" of traffic dynamics.

B. Multi-Agent Systems

Multi-Agent System is a framework where multiple independent agents interact with a shared environment with an individual or shared objectives. Decision making is distributed across multiple agents based on local and global observations. The advantage with multi-Agent systems is that agents can make autonomous decisions and can adapt their policies with respect to other agents' behavior.

In the domain of traffic control, each intersection can be considered as an independent agent by selecting traffic light phases at a time interval of 1 second. In a simulated city flow environment where traffic signals are controlled at intersection level with local observation of queued vehicles on incoming lanes. The global reward is the sum of individual rewards from all intersections.

As the decisions taken by each agent affect the overall traffic of all intersections, agents should have coordination between themselves with dynamic change in the flow of vehicles.

Centralized Training and Decentralized Execution (CTDE) Framework:

We have utilized a CTDE framework for training our multi-Agent RL network where the experiences from all the agents are collected into a single replay buffer which is used to train a single shared policy network.

During training, all intersections push their experiences into a single replay buffer, and the gradients are updated on a single Q network. During the inference, each intersection inputs its own local observation into the trained network which picks greedy actions. This CTDE setup adds stability and builds coordination between the intersections.

Our DQN network consists of three fully connected hidden layers which take observation vector as input and maps it to a vector of Q values with one Q value per action. Additional normalization is applied after each hidden layer for stability during training.

Hidden Layer Dimensions: [128, 256, 128]

Multi Agent Replay Buffer: The experiences from all the agents are pooled into a single replay memory which are sampled uniformly. This replay buffer maximizes data efficiency because same experience is used many times during training, and it also breaks the temporal correlation between the states which stabilizes the training. This experience consists of (states, actions, rewards, next states and dones).

*1) **DQN**:* Single Agent DQN's work for single intersections. As we move to the bigger neighborhoods, Multiple agents are required to collaborate to reduce the waiting time of vehicles across different intersections. Hence, we implemented DQN on City Flow environment with 16 intersections with complex road network and dynamic traffic flow. In all the different experimentations, we have used Centralized Training and Decentralized Execution (CTDE) Framework where agents learn collaborative behaviors through a shared replay buffer while retaining the fully independent decision-making at runtime.

Environment: CityFlow environment with 16 non-virtual intersections with a fixed road network and dynamic flow. The traffic arrivals are randomized using varying engine seed.
Episodes: 500 with 1000 steps per episode.
Network Architecture: An input of 72-dime vector per intersection passed through three hidden layers with number of neurons (128, 256, 128) with alternate normalization layers with 9 discrete Q values as output.

Multi-Agent Replay Buffer:

- Capacity: 15,000 transitions
- Stores tuples of (states, actions, next_states, rewards, dones) for all 16 agents
- Batch size: 128
- Train every 4 environment steps
- Target-network update every 8 training steps
- Optimizer: Adam, learning rate $1 * 10^{-4}$
- Discount factor $\gamma = 0.8$
- Epsilon start = 1.0 and Epsilon Min = 0.25

Normalization of Rewards: In highly dynamic environments like traffic control, raw rewards can vary significantly from one time step to another (Due to peak traffic or sudden drop of traffic). Feeding these high magnitude rewards directly to the Q network destabilizes the learning curve. This is because large TD errors lead to aggressive weight updates and slow convergence. Hence, we maintained a running estimate of mean and variance of the rewards using exponential moving average with a momentum of $\alpha = 10^{-3}$:

$$\mu_{t+1} = \mu_t + \alpha\big(r_t - \mu_t\big), \qquad \sigma^2_{t+1} = \sigma^2_t + \alpha\big((r_t - \mu_t)^2 - \sigma^2_t\big).$$

Then, these rewards are standardized as

$$\hat{r}_t \;=\; \frac{r_t - \mu_t}{\sqrt{\sigma^2_t + \varepsilon}}, \qquad \text{where } \varepsilon = 10^{-8}.$$

By keeping the reward inputs roughly zero-mean and unit-variance, the Q-value targets remain on a consistent scale throughout training. This leads to better performance, faster convergence and stable training.

Gradient Clipping: Deep Neural networks suffer from exploding gradients problem due to large TD errors which may make the training unstable. Hence, we rescale the gradient vector whenever its Euclidean norm exceeds a threshold. The maximum update of any parameter is capped, which prevents sudden long jumps in weights which stabilizes

the training. We have implemented gradient clipping with a threshold of 1.

*2) **Double DQN**:* To address the overestimation bias observed in standard DQN, we implemented the Double DQN algorithm[4]. In vanilla DQN, both selection and evaluation of the next-step action use the same target network, so errors can pile up which lead to overestimation of locally optimal signals by the agent.

In double DQN, we use the following update mechanism

$$y \;=\; r \;+\; \gamma\, Q_{\text{target}}\Big(s',\, \arg\max_{a'} Q_{\text{online}}(s', a')\Big),$$

The online network chooses the best next action, and the target network gives its value. This prevents the overestimation bias added by vanilla DQN. The training configuration is as follows:

- Batch size: 128
- Replay buffer size: 15 000 transitions (state, action, next state, reward, done)
- Discount factor $\gamma = 0.8$
- Exploration: ϵ-greedy, $\epsilon_0 = 1.0 \rightarrow \epsilon_{min} = 0.25$ decayed exponentially over 500 episodes
- Training frequency: update network every 4 environment steps
- Target network update: copy weights every 8 episodes.
- Maximum number of steps per Episode: 1000 steps
- Optimizer: Adam, learning rate $1 * 10^{-4}$
- Loss: mean-squared error

*3) **Dueling DQN with GAT** :* Here, we used Neighbor Aware reward function. The original reward, which is the negative sum of waiting vehicles at each intersection encourages the agents to focus on their own queues. In a connected city with multiple intersections, clearing one intersection only pushes the traffic to the neighboring intersections which causes congestion. Hence, we used a different reward function to promote collaboration between multiple agents which is neighbor aware reward.

Local waiting vehicles at intersection i, summing over its incoming lanes.

$$W_i = \sum_{\ell \in \mathcal{L}_i} \text{waiting}(\ell)$$

Original Reward which only considers local queues

$$r_i^{\text{orig}} = -W_i$$

Neighbor aware reward

$$r_i = -\Big(W_i + \alpha \sum_{j \in \mathcal{N}(i)} W_j\Big)$$

where N(i) is the set of adjacent intersections and alpha balances how much weight should be given to neighboring congestion. By penalizing both its own queue and a fraction of its neighbors' queues, each agent is driven to act not just selfishly but collaboratively, smoothing traffic over the entire network.

Dueling DQN: The dueling-DQN[5] architecture augments standard Q-learning by decomposing the estimated action-value Q(s, a) into two separate components:

1) State-value V(s), representing how good it is to be in state s regardless of action, and
2) Advantage A (s, a), capturing how much better taking action a is compared to the other actions in state s.

This separation helps the network learn which states are valuable even when the choice of action has little effect, and it reduces needless variance in the Q-estimates when one action is only marginally better than another.

After the shared network layers, it is split into two heads for estimating V(s) and A(s,a) which are combined to estimate the Q values as:

$$Q(s,a) \;=\; V(s) \;+\; \Big(A(s,a) \;-\; \frac{1}{|\mathcal{A}|} \sum_{a' \in \mathcal{A}} A(s,a')\Big).$$

Graph Attention Network: Intersections in a road network can naturally form a graph, where edges connect neighboring intersections. GAT[6] processes each node's local features while attending to its neighbors with learned, context-dependent weights.

By stacking two GAT convolutional layers, our model produces rich node embeddings that capture the local state of each intersection and main aspects of its neighbors' congestion patterns. Passing these embeddings into the dual heads ensures that each agent's Q-value reflects both its own queue and the broader network context, driving truly collaborative control.

Network Architecture: Two GAT convolutional layers stacked together with 4 and 1 attention heads respectively. Value Function and Advantage function as dual outputs with forward pass estimating the Q values from these two functions.

Graph Construction: GAT operates on graphs which are constructed from the city Flow Road network file where the real intersections are extracted. For every road linking two intersections, both forward and reverse directed edges so that information can flow in both directions. Self-loops are added to ensure that each node attends to its own embedding.

The training configuration for dueling DQN with GAT network is as follows:

- **Batch size:** 128

- **Replay buffer size:** 15 000 transitions (state, action, next state, reward, done)
- **Discount factor** (γ)**:** 0.8
- **Exploration:** ϵ-greedy, $\epsilon_0 = 1.0 \rightarrow \epsilon_{min} = 0.25$ decayed exponentially over 500 episodes
- **Training frequency:** update network every 4 environment steps
- **Target network update:** copy weights every 8 episodes.
- **Maximum number of steps per Episode:** 1000 steps
- **Optimizer:** Adam with a learning rate of 1e-4.
- **Loss:** mean-squared error between Bellman target and Q-network output

*4) **Prioritized Experience Replay (PER [7]) with Dueling DQN**:* In a standard replay buffer, every transition is considered equally important, but in most cases, some experiences carry large TD errors. If these experiences are sampled more often, the training can be accelerated. Each stored transition is assigned a priority which is equal to the TD error.

As the PER stores and updates its buffer with important experiences, we have reduced the replay buffer size to 10000 (from 15000 in the previous method) as fewer quality experiences are sufficient for learning.

*5) **Dueling Double DQN with PER**:* To make the learning more stable and achieve better results, we have converted the PER + Dueling DQN (GAT architecture) to double DQN. The training configuration is the same as the previous method where DQN is converted to double DQN.

IV. Results

A. DQN on Single Agent System

Epsilon Decay Graph

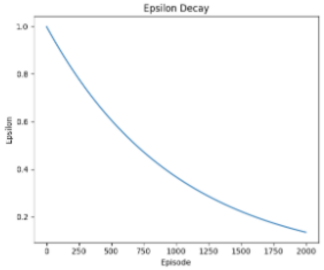

Fig. 4. Epsilon Decay after Implementing DQN on single agent system

Epsilon declines gradually starting at 1.0 initially and reaching a value of about 0.1 at episode 2000. We encourage proper exploration initially and eventually take greedy actions at the end of the episode.

Total Reward Per Episode Graph:

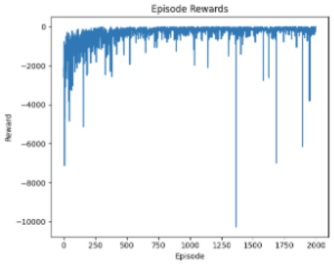

Fig. 5. Total reward per episode graph

We can observe that the agent is learning gradually, starting with highly negative rewards which improve significantly and stabilize close to zero over 2000 episodes. There are occasional negative spikes that are observed.

B. DQN on Multi Agent System

In the first episode, the total return is roughly –50,00,000, which corresponds to the total vehicle-seconds lost under a random traffic-signal policy. Over the next 500 episodes, the agents learn to reduce congestion and gradually improve that return to about –600 000. You'll notice, however, that the learning curve has sudden dips. These drops likely come from three main causes:

1) Overestimation bias: Q-learning can overvalue certain actions, so the policy occasionally chooses traffic-signal timings that look good on paper but actually perform poorly in practice.
2) Randomness in Traffic: Each episode uses a different random seed, so some traffic patterns like temporary surges or unusual flow, challenge the policy in ways it hasn't seen before, leading to brief performance collapses.
3) Unbounded reward scale: Since the reward is the negative sum of waiting times, extreme traffic in a single timestep can dominate the overall return and cause sharp downturns when they occur.

Fig. 6. Total Rewards for DQN Training on MultiAgent

Evaluation of the Random Policy (1000 Steps):

- Total steps executed: 999
- Total reward accumulated: –5475168.0
- Vehicles completed routes: 2316
- Still active vehicles: 784
- Average waiting time per vehicle: 55.12 s

- Average travel time (all vehicles): 187.42 s

Evaluation of Learned DQN Policy (1000 Steps Greedy Execution):

- Total steps executed: 999
- Total reward accumulated: –1000179.0
- Vehicles arrived: 2535
- Active vehicles remaining: 580
- Average waiting time per vehicle: 14.93 s
- Average travel time (all vehicles): 156.56 s

Under a random signal schedule, total waiting time is more than five times higher than with the trained DQN policy. Average waiting time is reduced from 55.12s to 14.93s eventually reducing the Average travel time of all the vehicles.

C. *Double DQN*

Fig. 7. Multi-Agent DQN Training

The total return rises more smoothly with less severe downward spikes compared to vanilla DQN, indicating more consistent policy gains over episodes.
It reduced the overestimation bias by separating the action selection and evaluation resulting in optimal Q values. Early episodes show a comparably rapid climb in return, implying that Double DQN extracts more reliable learning signals per interaction in this multi-agent setting.

Evaluation of Learned Double DQN Policy (1000 Steps Greedy Execution):

- Total steps executed: 999
- Total reward accumulated: -557334.0
- Vehicles arrived: 2558
- Active vehicles remaining: 557
- Average waiting time per vehicle: 10.79s
- Average travel time (all vehicles): 153.44s

D. *Dueling DQN with GAT :*

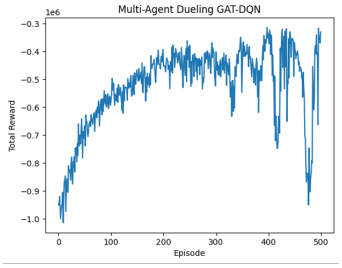

Fig. 8. Multi-Agent Dueling GAT-DQN

During the initial 200 episodes, the total return rises quickly from around $-1 * 10^6$ to $6 * 10^5$.
The network learns queue clearing strategies across the 16 intersections. The attention mechanism learns the long-range dependencies about the impact of signals at intersection A to delays at other intersections. There is a dip in performance after episode 300 due to the following reasons:

- Overestimation Bias
- Traffic Flow randomness
- Reward Mechanism: Although neighbor aware reward encourages collaboration, gradient updates oscillate if the local advantage of one intersection conflicts with its neighbor's queue reduction.

Evaluation of Learned Dueling DQN Policy with GAT Convolutional Layers (1000 Steps Greedy Execution)

- Total steps executed: 999
- Total reward accumulated: -279820.2
- Vehicles arrived: 2543
- Active vehicles remaining: 572
- Average waiting time per vehicle: 13.24s
- Average travel time (all vehicles): 156.10s

E. *Prioritized Experience Replay (PER) with Dueling DQN:*

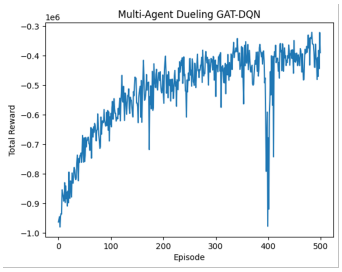

Fig. 9. Multi-Agent Dueling with PER

With the help of the Prioritized Experience replay, the training is much more stable than the basic Dueling DQN. Although, a sudden dip in the total reward is observed, which is still there because of the overestimation bias and the randomization of traffic during the training. The correction of importance sampling recovered quickly after these spikes.

Evaluation of Learned Dueling DQN Policy (GAT Convolution) with PER (1000 Steps Greedy Execution):

- Total steps executed: 999
- Total reward accumulated: -294144.0
- Vehicles arrived: 2545
- Active vehicles remaining: 570
- Average waiting time per vehicle: 14.2s
- Average travel time (all vehicles): 157.20s

F. Dueling Double DQN with PER

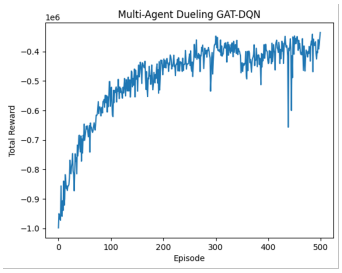

Fig. 10. Multi-Agent Dueling Double GAT-DQN with PER

With Double DQN, the overestimation bias is reduced which made the learning more stable than the previous approach.

Evaluation of Learned Dueling Double DQN Policy (GAT Convolution) with PER (1000 Steps Greedy Execution):

- Total steps executed: 999
- Total reward accumulated: -267640.8
- Vehicles arrived: 2540
- Active vehicles remaining: 575
- Average waiting time per vehicle: 12.9s
- Average travel time (all vehicles): 155.27s

V. Conclusion

TABLE I
PERFORMANCE OF DQN-BASED APPROACHES ON THE 16-INTERSECTION CITYFLOW NETWORK.

Approach	Inter-sections	Avg. Waiting Time (s)	Replay Buffer	Reward Function	Total Reward (1 000 steps)
Single-Agent DQN	1	—	10 000	Standard	—
Multi-Agent DQN	16	14.93	15 000	Standard	−1 000 179
Double DQN	16	10.79	15 000	Standard	−557 334
Dueling DQN	16	13.24	15 000	Neighbor-aware	−279 820
Dueling DQN + PER	16	14.20	10 000	Neighbor-aware	−294 144
Dueling Double DQN + PER	16	12.90	10 000	Neighbor-aware	−267 641

Standard Reward is the negative number of vehicles waiting per intersection.

- The Dueling Double DQN Network with Prioritized Experience Replay is the best performing approach with average waiting time of 12.9s across 1000 steps.
- Neighbor Aware Reward is used to encourage collaborative signaling strategy along with GAT convolutional architecture where the attention heads model the long-range dependencies between the intersections.
- The double DQN mechanism reduces the overestimation bias which is the key problem in DQN.
- Prioritized Experience Replay stabilizes the training with estimating the importance of experiences in the replay buffer.
- Dueling DQN improves results by estimating the Q values using the value function and the advantage function from the neural network.**cloudflareDomains2023**

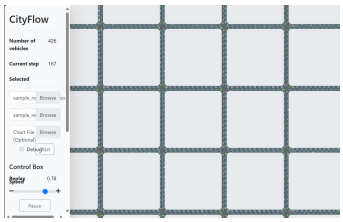

Fig. 11. CityFlow GUI snapshot for the 4×4 grid scenario(16 signalised intersections

The above image shows the lightweight visualisation tool we built on top of the CityFlow replay engine to observe our agents in real time. The 4*4 grid in the centre represents the 16 signalised intersections used in our experiments; coloured dots on each approach leg indicate the current traffic-light phase, while moving vehicle icons (not visible in this paused frame) trace individual trajectories through the network. The left-hand control panel streams live statistics—including the number of active vehicles, the current simulation step, and playback speed and lets us load roadnet, and flow without restarting the simulator. This interface proved invaluable for qualitative debugging (e.g., spotting deadlocks or uneven queue growth) and for visually confirming that the learned policies create coordinated "green waves" across multiple intersections.

VI. Contribution

All we three made equal contributions to the project formulation, implementation, experimentation, and writing of this project report.

References

[1] C. Developers, *Cityflow: A multi-agent traffic simulator*, https://cityflow.readthedocs.io, 2020.

[2] M. V. Dijck, *Gym-cityflow*, https : / / github . com / MaxVanDijck/gym-cityflow, 2021.

[3] V. Mnih, K. Kavukcuoglu, and D. Silver, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015. [Online]. Available: https://www.nature.com/articles/nature14236.

[4] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proceedings of the 30th AAAI Conference on Artificial Intelligence*, 2016, pp. 2094–2100. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/10295.

[5] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas, *Dueling network architectures for deep reinforcement learning*, https://arxiv.org/abs/1511.06581, 2016.

[6] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Lió, and Y. Bengio, *Graph attention networks*, https://arxiv.org/abs/1710.10903, 2018.

[7] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, *Prioritized experience replay*, https://arxiv.org/abs/1511.05952, 2016.